

COMP 3004 Winter 2026

HintonMarket Deliverable 2

Due: Thursday, April 2, 2026, at 23h59

Collaboration: This deliverable must be completed by registered teams

Overview

Your team will extend the HintonMarket system by implementing some of the Market Operator functionality (outlined in Part 2) and persistent data storage. You will also update the system decomposition diagram.

This deliverable focuses on system decomposition principles (coupling and cohesion), implementing some Market Operator features, and transitioning from in-memory storage to SQLite persistence.

This deliverable builds upon Deliverable 1 and the HintonMarket Project Overview document.

Part 1 – System Decomposition Diagram (20 marks)

Requirements

Students must submit **two versions** of the system decomposition diagram in a single PDF document:

1. **D1 Version** – The original system decomposition diagram submitted for Deliverable 1.
2. **Updated Version** – A revised diagram reflecting the current state of your code implementation.

Both diagrams must show only class names (no attributes or operations). Classes must be grouped into subsystems, and all associations and directionality between individual classes must be shown.

Written Explanation

In addition to the two diagrams, your document must include a written explanation covering the following:

- **What changed?** Describe the differences between your D1 diagram and your updated diagram. What classes, subsystems, or associations were added, removed, or reorganized?
- **Impact on cohesion and coupling.** Did your changes improve, worsen, or maintain cohesion and coupling? Explain your reasoning.
- **Architecture.** Identify the architecture your system decomposition diagram is based on. Did you change your architecture from your D1 solution? If so, explain why.

Submission for Part 1

Submit a PDF document containing both diagrams, each with a descriptive figure title (e.g., *"Figure 1: HintonMarket D1 System Decomposition"* and *"Figure 2: HintonMarket Updated System Decomposition Reflecting Current Implementation"*), followed by your written explanation.

Part 2 – HintonMarket Vertical Prototype (80 marks)

Persistent Data Storage Requirements

Your system must now use **SQLite** for data persistence. All data must be **both** (1) stored in objects in memory, and (2) stored in and retrieved from an SQLite database. If the SQLite database is not used, then a maximum of 20 marks out of 80 can be earned.

Database Requirements:

- **Create tables for your application, such as (not exhaustive):**
 - MarketSchedule
 - Users (vendors, market operator, system administrators)
 - Waitinglists
 - Any other necessary table
- **Object-Relational Mapping:**
 - Demonstrate clear mapping between your object-oriented domain model and relational database tables. Any query must create a corresponding object in

memory. **For instance**, if you are searching for a vendor in the database, the query results must be used to create the vendor object in memory.

- **The database must persist between program executions:**
 - When the program restarts, all market date items, user accounts, and waitlist items (other data you require to persist) must be restored from the database.
 - The system should **not** reset to default data on restart unless the database file is deleted or does not exist.

Default System Data

Your submission **must include a pre-populated SQLite database file** with the following initial data:

- **MarketSchedule** must include all of the following:
 - 4 weeks of markets are available for vendors to book.
 - A maximum of 2 food vendors and 2 artisan vendors market stalls are available per week (not the 20 food vendors and 10 artisan vendors mentioned in the HintonMarket overview document). *This will allow for easier testing the waitlist for a market stall.*
- **10 unique users**, including all the following:
 - **4 unique food vendors**
 - With vendor information, compliance documents information, name, and address
 - **4 unique artisan vendors**
 - With vendor information, compliance documents information, name, and address
 - **1 market operator**
 - **1 system administrator**

Important: The database file must be included in your submission with all tables created and populated with this default data. The TA should be able to run your program immediately without any initialization steps.

Functional Requirements

Carry Forward from D1

- All vendor functionality from Deliverable 1 must continue to work and **must use the SQLite database to store data.**

New Market Operator Features (Must Implement)

These features must be available once the Market Operator enters into the system from the enter page.

1. Book a stall for a specified market date on behalf of a vendor
2. Cancel a stall booking on a vendor's behalf.
3. On behalf of the vendor, remove them from a specified market date waiting list.

All three changes done on behalf of the vendor must be reflected in the vendor dashboard when the vendor returns to the HintonMarket.

In all the cases above, assume the vendor has contacted the Market Operator with the details.

Business Rules

- Waitlists are organized by vendor category (Food or Artisan) AND by market week
- Maximum of 8 active waitlists (4 weeks × 2 categories)
- Each waitlist follows first-come, first-served (FIFO) order
- When a stall becomes available via cancellation, the system notifies the first vendor in the appropriate waitlist queue (matching both category and week)
- Vendors can book only one market stall date at a time.

Excluded from this Prototype:

- No stall payment processing
 - No booking deadline enforcement (Wednesday 11:59 PM booking window)
 - No cancellation deadline enforcement (Wednesday 12:00 PM cancellation deadline)
 - No refund processing
-

Grading Breakdown

Part 1- System Decomposition Diagram 20 marks

Part 2 - Database and Market Operator Features Implementation 80 marks

Implementation Requirements

(This section defines the technical environment, design standards, and quality attributes that constrain how the HintonMarket system must be built.)

- You must build upon your own team's D1 implementation when completing this deliverable. However, all code must originate from your team's work. Using or modifying code from another team's D1 or any outside source is not allowed and will be reported as an academic integrity offence.
- The following requirements specify the **technical environment, design expectations, and functional behaviour** of the HintonMarket system. Your implementation must meet

these standards to ensure it operates correctly within the course VM and demonstrates appropriate object-oriented design.

- The Linux Ubuntu platform, as provided in the official course virtual machine (VM), will be used as the test bed for evaluating your code.
 - All source code must be written in C++, and it must be designed and implemented at the level of a 3rd-year undergraduate Computer Science student, following standard UNIX programming conventions, and using advanced OO programming techniques such as polymorphism and design patterns.
 - Only libraries already provided as part of the course VM may be used.
 - The HintonMarket must provide a graphical user interface built with Qt widgets. In general, user features should be easily navigable, either as menu items and/or pop-up menus. The look-and-feel of the HintonMarket system should be professional and consistent with commercially available UIs.
 - SQLite must be used for data persistence. The SQLite library is available in the course VM.
 - The Market Operator interface should clearly distinguish market operator specific features from vendor features.
 - The HintonMarket system will run on a single host.
 - Data storage for Deliverable 2 must use SQLite for persistence.
-

Submission Requirements

(This section defines how your project must be packaged, extracted, built, and run for grading.)

- Your submission must be fully turnkey; the TA must be able to extract the zip file, then import, build, and run your application in Qt Creator on the course VM in under three (3) minutes

.Requirements:

- The TA must be able to **extract your ZIP file** and open the project directly in **Qt Creator**.
- The TA must be able to **build and run the application within Qt Creator** without changing any file paths, settings, or configuration options.
- **You must include a pre-populated SQLite database file** (hintonMarket.sqlite3) with the required default data (as specified above). The database file must be in the correct location so the application can load it immediately when it runs.
- **All required files** (source code, .pro, resources, and data) must be included in your zip file. The TAs will open your project in Qt Creator, loading the .pro. If no .pro file is available, then the TAs will not be able to load and run your project, and you will receive 0 marks for the functional implementation.
- **No manual installation of libraries**, environment setup, or directory relocation is permitted.

- All setup and usage instructions, as well as the **application vendor, market operator and system administrator user names**, must be **clearly documented** in your README file.
 - The project must compile and execute correctly **as described under Implementation Requirements**.
 - Projects that do not open, build, and run in Qt Creator as described **will not be graded**.
-

Format Requirements

- File Naming and Format: Submit as PDF using naming convention team_#_D2.pdf (# is your group number in BrightSpace). Documents must be typed, legible, and professional, including a cover page (with title, date, team number and team member names, and student numbers).
- UML Diagrams: All UML diagrams must be created using professional drawing tools. Hand-drawn diagrams will not be accepted. Note that the diagram must be embedded in the PDF document and **not** as a link. If a link to a diagram is given in the PDF document, then you will receive 0 for the System Decomposition Diagram and explanation.
- Documents that do not conform to these specifications will earn a grade of zero.
- Coding deliverables must be provided as a single zip file containing all source code and data files, as well as installation, build, and launch instructions, and names of user accounts in a README file and a system decomposition diagram PDF. Do not provide object files and project executables as part of your submission. Zip file naming convention: team_#_D2.zip.
- Your coding deliverable must be submitted as a single zip file (team_#_D2.zip, where # is your team number) containing:
 - All source code and data files
 - Qt project file (.pro)
 - Database SQLite file
 - README file with:
 - Installation, build, and launch instructions
 - User account names for testing application
 - System decomposition diagrams (PDF) and explanation
 - **Do not include:** object files or compiled executables in your submission.